
Java Master Perfect Guide

V 1.00

佐藤 明彦

20.関数型インターフェース

関数型インターフェースとは、抽象メソッドを1つしか持たないインターフェースのことです。ただし、default メソッドや static メソッドは持っていても構いません。

Runnable という Java のクラスライブラリに元々あるインターフェースを見てみましょう。run（実行する）という抽象メソッドがあります。

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

抽象メソッドは1つだけなので、FunctionalInterface アノテーションを宣言の前（1行目）に指定しています。

アノテーションは注釈という意味です。抽象メソッドが1つだけなら関数型インターフェースになるのですが、FunctionalInterface アノテーションで明示的に宣言するのが良いでしょう。

20.1.関数型インターフェースの実装

Runnable インターフェースを、まずは**普通**に実装していきます。implements キーワードを使います。

Example.java

```
1 public class Example implements Runnable {
2     @Override
3     public void run() {
4         System.out.println("Example を実行しました");
5     }
6 }
```

クラス名は Exapmle としてみました。run()メソッドをオーバーライドしています。

このメソッドは Override だよと明言しているのが Override アノテーションです。引数の数の指定やデータ型の記述ミスで、予期せぬオーバーライドやオーバーライド失敗を防ぐための注釈です。なくても良いのですが、書いておいた方が安全です。

処理の中では、Example を実行しましたメッセージを println しています。

Exapmle クラスを実行する側の main メソッドについて、見ていきたいと思います。

Main33.java

```
1  public class Main33 {  
2      public static void main(String[] args) {  
3          Runnable e = new Example();  
4          e.run();  
5      }  
6  }
```

Example クラスを new して、Runnable インターフェース型の変数 e に代入しています。
e.run()として実行しています。

Runnable インターフェースはマルチスレッドのプログラムを作るためのものなので、本来このようなやり方はしないのですが、伝わりやすくしようと、今回採用しています。

実際に実行すると、コマンドラインに「Example を実行しました。」メッセージが出力されるわけです。

実行結果

```
Example を実行しました
```

でも、これをやりたいだけなのに、Runnable インターフェースも含めると 3 つのファイルを使っているのです。

- Runnable インターフェース
- Runnable インターフェースを実装するクラス
- main メソッドを持つクラス

この中の、Runnable インターフェースを実装している Example クラスって、本当に必要なのでしょうか？ということで、次のような記述ができるようになっているのです。

内部クラスにする

Example クラスを、main メソッドを持つクラスの中に宣言します。クラスの中に記述されているクラスを内部クラスと呼びます。

Main34.java

```
1 public class Main34 {
2     private static class Example implements Runnable {
3         public void run() {
4             System.out.println("Example を実行しました");
5         }
6     public static void main(String[] args) {
7         Runnable e = new Example();
8         e.run();
9     }
10 }
```

実行結果

Example を実行しました

こうすると、main メソッドの中から Example クラスを呼び出すことができます。main メソッドの中は先ほどと同じです。

匿名クラスにする

一時的な使い捨てのクラスとしての用途です。

Main35.java

```
1 public class Main35 {
2     public static void main(String[] args) {
3         Runnable e = new Runnable() {
4             @Override
5             public void run() {
6                 System.out.println("main の中で run を実行！");
7             }
8         };
9         e.run();
10     }
```

実行結果

main の中で run を実行！

class の宣言も、クラス名もありません。Runnable インターフェースを new しています。実際にはインターフェースを new しているのではなく、匿名クラスを new しているのがポイントです。

内部クラスの時よりも、書式が簡潔になりました。private なスタティック class を作る必要もありません。

これを匿名クラスと呼びます。

21.ラムダ式

ラムダ式とは、「匿名クラスの表記を省略したもの」です。コードを簡潔に記述するための言語仕様で、ラムダ式を使うことでメソッドを簡単に定義し、その場で使用することができます。それでは、どのように省略していくのか、みていきましょう。

21.1.ラムダ式に変換する 10 の手順

1. インターフェースの表記を省略
2. 修飾子の表記を省略
3. 抽象メソッドの戻り値の型を省略
4. 抽象メソッドの名前を省略
5. 抽象メソッドの引数の型を省略
6. return を省略
7. 抽象メソッドの{}と;を省略
8. 匿名クラスの{}を省略
9. new 演算子を省略
10. アロー演算子を記述

最後だけ記述です。

いっぱいありすぎて混乱するかもしれませんが、この 10 の手順に従えば、匿名クラスの宣言がラムダ式になります。

省略前のソースコードは以下のようになります。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new Runnable() {
4              @Override
5              public void run() {
6                  System.out.println("main の中で run を実行！");
7              }
8          };
9          e.run();
10     }
11 }
```

1. インターフェースの表記を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new {
4              @Override
5              public void run() {
6                  System.out.println("main の中で run を実行！");
7              }
8          };
9          e.run();
10     }
11 }
```

new 演算子のところの Runnable() を削除しました。

2. 修飾子の表記を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new {
4              @Override
5              void run() {
6                  System.out.println("main の中で run を実行！");
7              }
8          };
9          e.run();
10     }
11 }
```

run メソッドの public を削除しました。

3. 抽象メソッドの戻り値の型を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new {
4              @Override
5              run() {
6                  System.out.println("main の中で run を実行！");
7              }
8          };
9          e.run();
10     }
11 }
```

void を削除しました。

4. 抽象メソッドの名前を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new {
4              @Override
5              () {
6                  System.out.println("main の中で run を実行！");
7              }
8          };
9          e.run();
10     }
11 }
```

run を削除しました。

5. 抽象メソッドの引数の型を省略する。

このサンプルの場合、run メソッドには引数がないので、省略できるものはありません。引数がある場合は、型を削除してください。

6. return を省略する。

戻り値もないので省略できるものはありません。return 文がある場合は、消去してください。

7. 抽象メソッドの{}と;を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new {
4              @Override
5              ()
6                  System.out.println("main の中で run を実行！")
7
8          };
9          e.run();
10     }
11 }
```

()と System.out.println の命令文だけが1行残りました。

8. 匿名クラスの{ }を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e = new
4              @Override
5              ()
6              System.out.println("mainの中でrunを実行！")
7
8          ;
9          e.run();
10     }
11 }
```

セミコロンだけが残りました。

9. new 演算子を省略する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e =
4              @Override
5              ()
6              System.out.println("mainの中でrunを実行！")
7
8          ;
9          e.run();
10     }
11 }
```

10. アロー演算子を記述する。

```
1  public class Main {
2      public static void main(String[] args) {
3          Runnable e =
4              @Override
5              () ->
6              System.out.println("mainの中でrunを実行！")
7
8          ;
9          e.run();
10     }
11 }
```

()の後ろに記述します。

まとめるとこうなります。

Main36.java

```
1  public class Main36 {  
2      public static void main(String[] args) {  
3          Runnable e =  
4              () -> System.out.println("ラムダ式で main の中で run を実行！");  
5          e.run();  
6      }  
7  }
```

実行結果

ラムダ式で main の中で run を実行！

ラムダ式の部分はココです。

() -> System.out.println("ラムダ式で main の中で run を実行！")

とっても簡潔な記述になりました。作成したクラスもこのクラスのみです。ラムダ式は、Stream API と呼ばれる言語仕様でよく利用されます。

20.Stream API

20.1.関数型プログラミングについて

本題に入る前に、関数型プログラミングについて触れておきたいと思います。関数型プログラミングとは、「計算を関数の連鎖で行うコーディングスタイル」のことです。

- Haskell
- Scala
- etc...

などのプログラミング言語がそれにあたります。

- Java
- Python
- JavaScript

といったよく知られた言語は、関数型プログラミングの概念をサポートしていますが、「関数を実行する言語」です。

関数型プログラミングは「式（関数）を評価する」と表現します。

20.2.Stream API とは？

Stream API とは、関数型プログラミングのアプローチを取り込んだライブラリのこと、クラス群またはインターフェース群のことです。java.util.stream パッケージのクラス群がそれにあたります。代表的なものに、Stream インターフェースがあります。

20.2.1.Stream インターフェース

Stream とは、「順次および並列の集約操作をサポートする要素のシーケンス」と Java のリファレンスに記載されています。

以下は、リファレンスの抜粋です。

```
public interface Stream<T>  
    extends BaseStream<T, Stream<T>>
```

順次および並列の集約操作をサポートする要素のシーケンスです。次の例は、*Stream* と *IntStream* を使用する集計操作を示したものです。

```
int sum = widgets.stream()  
    .filter(w -> w.getColor() == RED)  
    .mapToInt(w -> w.getWeight())  
    .sum();
```

<https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>

シーケンスとは、順番に並んだデータの集合を指します。

- 配列
- List
- Stream

Stream には、配列や List にはない、順次および並列の集約操作がサポートされています。

順次の集約操作とは？「要素を 1 つずつ順番に評価してその結果を集約すること」です。
filter()、map()、forEach()などのメソッドの事です。

並列の集約操作とは？「1 つずつではなく、各要素を同時に並行して処理をする」という事です。
parallelStream というメソッドを使うと、それ以降の集約操作が並列になります。

20.2.2. Stream API のラムダ式

ラムダ式とは、「匿名クラスで実装した関数型インターフェースの記述を省略したもの」でした。

ラムダ式に変換する 10 の手順を、逆の手順で匿名クラスに戻したいと思います。以下の例を使って、ラムダ式の意味を紐解いていきましょう！

Main37.java

```
1  import java.util.Arrays;
2  import java.util.List;
3
4  public class Main37 {
5      public static void main(String[] args) {
6          List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
7          nums.stream().filter(i -> i % 2 == 0)
8              .forEach(i -> System.out.println(i * 2));
9      }
10 }
```

この例は、1 から 5 の数値の羅列をリストに変換して、変数 nums に代入しています。nums から stream を取得して数珠繋ぎ風に filter() メソッドを呼び出しています。このような繋ぎ方（呼び出し方）をメソッドチェーンと言います。これが川のようにワンライナー（1 行）で流れていくので Stream なわけです。

filter

filter メソッドのリファレンスを見てみましょう。

Stream<T> filter(Predicate<? super T> predicate)

このストリームの要素のうち、指定された述語に一致するものから構成されるストリームを返します。

これは中間操作です。

パラメータ: predicate - 各要素を含めるべきか判定する目的で各要素に適用する、非干渉でステートレスな述語
戻り値: 新しいストリーム

<https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html#filter-java.util.function.Predicate->

引数の型は、プレディケートになっています。これは関数型インターフェースです。

インターフェース *Predicate<T>*

型パラメータ: T - 述語の入力の型

関数型インターフェース: これは関数型インターフェースなので、ラムダ式またはメソッド参照の代入先として使用できます。

<https://docs.oracle.com/javase/8/docs/api/java/util/function/Predicate.html>

抽象メソッドは、test というメソッドになります。

test

boolean test(T t)

指定された引数でこの述語を評価します。

パラメータ:

t - 入力引数

戻り値:

入力引数が述語に一致する場合は *true*、それ以外の場合は *false*

<https://docs.oracle.com/javase/jp/8/docs/api/java/util/function/Predicate.html#test-T->

test メソッドには、引数 *t* を評価する処理をオーバーライドしてください、という意味になるわけです。戻り値は評価した結果を返す、boolean 型になっています。

ラムダ式を、匿名クラスにリバーズしていきましょう。

```
1 public class Main {
2     public static void main(String[] args) {
3         List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4         nums.stream().filter(i -> i % 2 == 0)
5             .forEach(i -> System.out.println(i * 2));
6     }
7 }
```

まず、ラムダ式を消します。

```
1 public class Main {
2     public static void main(String[] args) {
3         List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4         nums.stream().filter()
5             .forEach(i -> System.out.println(i * 2));
6     }
7 }
```

Predicate インターフェースを new して匿名クラスを宣言していきます。

```
1 public class Main {
2     public static void main(String[] args) {
3         List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4         nums.stream().filter(new Predicate<Integer>() {
5
6             }).forEach(i -> System.out.println(i * 2));
7     }
8 }
```

変数 nums の要素、1、2、3、4、5 は数値なのでジェネリクス部分はラッパークラスの Integer 型にしてやります。匿名クラスの中身はまだ何もありません。

test メソッドの中身を実装します。test メソッドをオーバーライドします。

```
1 public class Main {
2     public static void main(String[] args) {
3         List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4         nums.stream().filter(new Predicate<Integer>() {
5             @Override
6             public Boolean test() {
7
8             }
9             }).forEach(i -> System.out.println(i * 2));
10    }
11 }
```

引数のデータ型は Integer、引数名は i としました。ここに 1、2、3、4、5 が順番に入ってくるというわけです。

test メソッドの中身を実装します。

Main38.java

```
1  import java.util.Arrays;
2  import java.util.List;
3  import java.util.function.Predicate;
4
5  public class Main38 {
6      public static void main(String[] args) {
7          List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
8          nums.stream().filter(new Predicate<Integer>() {
9              @Override
10             public Boolean test() {
11                 return i % 2 == 0;
12             }
13         }).forEach(i -> System.out.println(i * 2));
14     }
15 }
```

引数 *i* を 2 で割って余が 0 かを評価しています。偶数なら true、奇数なら false をリターンします。匿名クラスの完成です！先ほどのラムダ式はこういう意味だったのです。

forEach

forEach のリファレンスです。

forEach

void forEach(Consumer<? superT> action)

このストリームの各要素に対してアクションを実行します。

これは終端操作です。

この操作の動作は明らかに非決定論的です。並列ストリーム・パイプラインの場合、この操作は、ストリームの検出順序を考慮することを保証*しません*。保証すると並列性のメリットが犠牲になるからです。与えられた任意の要素に対し、ライブラリが選択した任意のタイミングで任意のスレッド内でアクションが実行される可能性があります。アクションが共有状態にアクセスする場合、必要な同期を提供する責任はアクションにあります。

パラメータ: *action* - 要素に対して実行する[非干渉]アクション

<https://docs.oracle.com/javase/jp/8/docs/api/java/util/stream/Stream.html#forEach-java.util.function.Consumer->

引数の型は Consumer インターフェース型です。

インターフェース *Consumer<T>*

型パラメータ:*T* - オペレーションの入力の型

既知のすべてのサブインターフェース:*Stream.Builder<T>*

関数型インターフェース:これは関数型インターフェースなので、ラムダ式またはメソッド参照の代入先として使用できます。

<https://docs.oracle.com/javase/jp/8/docs/api/java/util/function/Consumer.html>

関数型インターフェースなのでラムダ式に省略できます。

accept

void accept(T t)

指定された引数でこのオペレーションを実行します。

パラメータ:*t* - 入力引数

<https://docs.oracle.com/javase/jp/8/docs/api/java/util/function/Consumer.html#accept-T->

accept という 1 つの抽象メソッドを持っています。ラムダ式を、匿名クラスにリバースしていきましょう。

forEach メソッドのラムダ式を消去します。

```
1 public class Main {
2     public static void main(String[] args) {
3         List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4         nums.stream().filter(new Predicate<Integer>() {
5             @Override
6             public Boolean test() {
7                 return i % 2 == 0;
8             }
9         }).forEach();
10    }
11 }
```

Consumer インターフェースを new して匿名クラスを宣言していきます。

```
1  public class Main {
2      public static void main(String[] args) {
3          List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
4          nums.stream().filter(new Predicate<Integer>() {
5              @Override
6              public Boolean test() {
7                  return i % 2 == 0;
8              }
9          }).forEach(new Consumer<Integer>() {
10
11      });
12  }
13 }
```

匿名クラスの中身を書いていきましょう。

Main39.java

```
1  import java.util.Arrays;
2  import java.util.List;
3  import java.util.function.Consumer;
4  import java.util.function.Predicate;
5
6  public class Main39 {
7      public static void main(String[] args) {
8          List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
9          nums.stream().filter(new Predicate<Integer>() {
10              @Override
11              public Boolean test() {
12                  return i % 2 == 0;
13              }
14          }).forEach(new Consumer<Integer>() {
15              @Override
16              public void accept(Integer i) {
17                  System.out.println(i * 2);
18              }
19          });
20  }
21 }
```

accept メソッドを、 $i * 2$ して println する処理でオーバーライドしました。accept メソッドの戻り値は void なので return 文はありません。匿名クラスが完成しました。この例では、偶数 2 と 4 の倍、4 と 8 が出力されます。Stream API で評価されるラムダ式の省略表記を、匿名クラスに戻していくと、実装内容の意味がわかると思います！